Lesson Plan: Module 4

AI-Driven Classification Pipeline:
Preprocessing, Model Training, Evaluation,
and Real-Time Inference

An Educational Framework for
AI-Driven Signal Processing

Student:	Jude Eschete
Project Advisor:	Dr. Bernard Yett
Program:	M.S. Computer Engineering
Institution:	Stevens Institute of Technology
Date Range:	April 12 – April 25, 2026

This document presents the lesson plan for the fourth module of the capstone project. Students build the complete Machine Learning pipeline that consumes the labeled Radio Frequency dataset collected in Module 3, preprocess Field-Programmable Gate Array-extracted feature vectors, train and compare multiple classifiers for spreading factor detection, modulation classification, and emitter identification, quantify the simulation-to-hardware distribution shift through cross-domain transfer experiments, and deploy a real-time inference script that classifies live Field-Programmable Gate Array feature exports as they arrive.

JEschete@stevens.edu

Contents

1 Module Overview **1**
1.1 Context Within the Project . . . 1
1.2 Prerequisites . . . 2

2 Weekly Schedule **3**

3 Week 1: Data Preprocessing and Feature Engineering **4**
3.1 Objectives . . . 4
3.2 Dataset Loading and Validation . . . 4
3.2.1 Validation Checks . . . 4
3.3 Sentinel Value Handling . . . 5
3.4 Signed-Field Decoding . . . 5
3.5 Feature Selection by Classification Task . . . 6
3.5.1 Retained Features by Task . . . 7
3.6 Feature Scaling . . . 7
3.7 Preprocessing Pipeline Implementation . . . 8

4 Week 2: Baseline Models and Classical Classifiers **10**
4.1 Objectives . . . 10
4.2 Baseline Classifiers . . . 10
4.3 Random Forest Classifier . . . 10
4.3.1 Hyperparameter Search Space . . . 10
4.3.2 Feature Importance Analysis . . . 10
4.4 Support Vector Machine (SVM) . . . 11
4.4.1 Hyperparameter Search Space . . . 12
4.5 Evaluation Methodology . . . 12
4.5.1 Metrics . . . 12
4.5.2 Evaluation Protocol . . . 12

5 Week 3: Neural Network Models and Transfer Experiments **15**
5.1 Objectives . . . 15
5.2 Multi-Layer Perceptron (MLP) . . . 15
5.2.1 Architecture . . . 15
5.2.2 Training Configuration . . . 15
5.3 One-Dimensional CNN (CNN-1D) . . . 16
5.3.1 Architecture . . . 16
5.4 Simulation-to-Hardware Transfer Experiment . . . 17
5.4.1 Experimental Protocol . . . 17
5.5 Performance-vs.-Condition Analysis . . . 18
5.5.1 Signal-to-Noise Ratio (SNR)-Stratified Accuracy . . . 18
5.5.2 Distance-Stratified Accuracy . . . 18
5.5.3 Beacon-Count-Stratified Accuracy . . . 18

6 Week 4: Real-Time Inference and Pipeline Integration **20**
6.1 Objectives . . . 20
6.2 Real-Time Inference Architecture . . . 20
6.2.1 Real-Time Constraint . . . 20
6.3 Inference Script Implementation . . . 21
6.3.1 Key Components . . . 21
6.3.2 Multi-Task Inference . . . 22
6.4 Live Demonstration Protocol . . . 22
6.5 Inference Latency Measurement . . . 23
6.6 Final Model Selection . . . 23

7 Complete Classification Pipeline Architecture **25**

8 Software Reference **26**
8.1 Hardware Requirements . . . 26

9 Assessment Rubric **26**

10 Recommended Resources **27**
10.1 Machine Learning Textbooks . . . 27
10.2 RF and Signal Classification References . . . 27
10.3 Software Documentation . . . 27
10.4 Domain Adaptation and Distribution Shift . . . 28

11 Transition to Module 5 **29**

Glossary **30**

1 Module Overview

Module 4 Learning Objectives

Upon completion of this module, the student (or future learner following this curriculum) will be able to:

LO-1 Load, clean, and preprocess the hardware-collected Radio Frequency (RF) dataset from Module 3, handling sentinel values, signed-field decoding, and feature scaling in a reproducible Python pipeline.

LO-2 Select and justify an appropriate feature set for each classification task, using exploratory data analysis and domain knowledge to identify informative features and remove redundant or leaking ones.

LO-3 Train, tune, and evaluate at least three classifier families—Random Forest, Multi-Layer Perceptron (MLP), and a one-dimensional Convolutional Neural Network (CNN-1D)—on spreading factor detection, modulation classification, and emitter identification tasks.

LO-4 Apply rigorous evaluation methodology: stratified train/validation/test splits, confusion matrices, per-class precision/recall/F1-score, and learning curves, avoiding data leakage and overfitting.

LO-5 Quantify the simulation-to-hardware distribution shift by training classifiers on engagement simulator data and evaluating on hardware-collected data, measuring the accuracy degradation and identifying which features contribute most to the domain gap.

LO-6 Analyze how classification accuracy varies as a function of signal-to-noise ratio (SNR), beacon count, and physical distance, producing performance-vs.-condition curves that characterize the system's operational envelope.

LO-7 Deploy a real-time inference script that receives live feature vectors from the Field-Programmable Gate Array (FPGA) via Universal Asynchronous Receiver/Transmitter (UART) and classifies each observation within one inter-packet interval, demonstrating end-to-end pipeline integration from RF transmission to classification output.

1.1 Context Within the Project

Module 4 is where the "AI-Driven" portion of the framework's title becomes concrete. Modules 1–3 built and validated the hardware platform, FPGA processing pipeline, and data collection infrastructure. This module consumes their output—a structured, labeled RF feature dataset—and transforms it into trained classification models that can identify what a beacon transmitted based solely on the FPGA-extracted feature vector.

This module is deliberately positioned *after* data collection, not concurrent with it. The framework's pedagogical design requires students to understand their data before modeling it. The EDA performed in Module 3 Week 4 already revealed feature distributions, correlations, and class balance. Module 4 builds on that understanding: preprocessing decisions (which features to scale, which to drop, how to handle sentinels) are informed by the EDA, and model interpretation (which

features drive classification) loops back to the hardware design decisions made in Modules 1–2.

The cross-module dependency chain reaches its full expression here: a beacon packet format decision in Module 1 determined which fields the FPGA parser extracts in Module 2, which features appear in the dataset collected in Module 3, and which classification tasks are feasible in Module 4. A student who changes the packet format in Module 1 must propagate that change through three downstream modules. This chain is the framework's central teaching instrument.

1.2 Prerequisites

Students beginning this module should have:

- Completed Module 3 with a merged, labeled RF dataset (`.csv`) containing ≥2,500 feature vectors across campaigns C1–C5 with a stratified train/val/test split.
- The Module 3 dataset summary document (data dictionary, class balance, known limitations).
- Python 3.10+ with the following libraries installed: `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `seaborn`, `torch` (PyTorch), `pyserial`.
- Familiarity with supervised classification concepts: features, labels, training, overfitting, cross-validation.
- Introductory exposure to neural networks (MLP architecture, backpropagation, loss functions).
- Access to the FPGA system (Nexys A7 running the Module 2 pipeline) for real-time inference exercises in Week 4.
- (Optional) Engagement simulator data from the project's early development phase, for the sim-to-real transfer experiment.

2 Weekly Schedule

The module spans two weeks (four working sessions), progressing from data preparation through model training and evaluation to live deployment.

Table 1: Module 4 weekly breakdown.

Week	Focus Area	Key Activities
1	Data Preprocessing & Feature Engineering	Load dataset; handle sentinel values and signed-field decoding; feature selection per classification task; normalization and scaling; verify train/val/test split integrity; build reusable preprocessing pipeline.
2	Baseline Models & Classical Classifiers	Train Random Forest and SVM classifiers for each task; hyperparameter tuning via grid search on validation set; evaluate on held-out test set; compute confusion matrices, per-class metrics, and feature importances.
3	Neural Network Models & Transfer Experiments	Train MLP and CNN-1D classifiers; compare against classical baselines; conduct sim-to-real transfer experiment; analyze accuracy vs. SNR, distance, and beacon count; generate performance-vs.-condition curves.
4	Real-Time Inference & Pipeline Integration	Implement live inference script; connect to FPGA UART export; classify feature vectors in real time; measure inference latency; final model comparison and selection; document complete ML pipeline.

3 Week 1: Data Preprocessing and Feature Engineering

3.1 Objectives

- Load the Module 3 dataset and verify its integrity against the data dictionary.
- Handle sentinel values, signed-field decoding, and data type conversions.
- Select task-specific feature sets that avoid label leakage.
- Implement feature scaling and normalization appropriate for each classifier family.
- Verify the stratified train/val/test split and ensure no data leakage across splits.
- Package the preprocessing steps into a reusable Python pipeline.

3.2 Dataset Loading and Validation

The first step is to load the merged CSV from Module 3 and verify it against the data dictionary. This is not merely a file-read operation—it is a data quality gate that catches errors introduced during collection, merging, or manual editing.

3.2.1 Validation Checks

Table 2: Dataset validation checks.

#	Check	Action on Failure
1	All expected columns present (21 columns per Module 3 Table 7).	Halt: schema mismatch.
2	No null or NaN values in feature columns.	Investigate; likely collection script bug.
3	`beacon_id` values $\in [1, 255]$.	Remove out-of-range rows; log count.
4	`mod_code` values $\in \{1, 2, 3\}$ (LoRa, FSK, OOK).	Flag unexpected codes.
5	`spread_factor` values $\in \{0, 7, 8, 9, 10, 11, 12\}$.	Flag unexpected values.
6	`rssi` values $\in [-140, 0]$ dBm (plausible range).	Clip or flag outliers.
7	`snr` values $\in [-20, 15]$ dB (SX1276 reported range).	Clip or flag outliers.
8	`split` column contains only `train`, `val`, `test`.	Halt: split assignment error.
9	Each class has ≥ 30 samples in the test split.	Warn: insufficient test data for that class.
10	No duplicate (`campaign_id`, `beacon_id`, `seq_num`) triples.	Remove duplicates; log count.

3.3 Sentinel Value Handling

The FPGA feature extraction logic (Module 2 Section 5) uses sentinel values for first-observation edge cases:

- **Inter-Arrival Time** = `0xFFFFFFFF` (4,294,967,295 ms) when a beacon is seen for the first time (no previous timestamp exists).
- **Received Signal Strength Indicator (RSSI) Delta** = `0x0000` (zero) on first observation.

These sentinels must be handled before training, as they are not meaningful signal measurements. Two approaches are appropriate:

1. **Drop first-observation rows:** Remove all rows where `obs_count` = 1. This is the simplest approach and loses only ∼5–15 rows (one per beacon per campaign run). It eliminates both sentinel values simultaneously.
2. **Replace with per-class median:** For inter-arrival time, replace the sentinel with the median inter-arrival time for that beacon's spreading factor and scheduling mode. This preserves rows but introduces imputed values that the model cannot distinguish from real measurements.

Recommendation

Use the drop approach (option 1) for this module. The dataset is small enough that losing ∼15 rows is inconsequential, and imputation introduces a subtle data integrity risk that is not worth the complexity for a teaching exercise. The key lesson is that the student recognizes the sentinel, traces its origin to the FPGA design decision in Module 2, and handles it explicitly rather than allowing it to silently corrupt the model.

3.4 Signed-Field Decoding

Several feature vector fields are signed quantities stored as unsigned bytes in the FPGA Block Random Access Memory (BRAM) and transmitted as raw bytes over UART:

Table 3: Signed-field decoding requirements.

Field	Raw Type	True Type	Decoding
`rssi`	uint8	int8	Two's complement: if value > 127, subtract 256.
`snr`	uint8	int8	Two's complement. SX1276 reports SNR in 0.25 dB steps; divide by 4 for dB.
`freq_error`	uint16	int16	Two's complement 16-bit signed integer (Hz).
`rssi_delta`	uint16	int16	Two's complement 16-bit signed delta.

Common Pitfall: Double Decoding

If the Module 3 Python collection script already performed signed decoding when writing the CSV, applying the decoding again in Module 4 will produce incorrect values. Check the Module 3 data dictionary to determine whether the CSV contains raw unsigned values or already-decoded signed values. The validation check in Table 2 row 6 (RSSI $\in [-140, 0]$) will catch this: if RSSI values appear as large positive numbers (e.g., 180), they are still unsigned and need decoding.

3.5 Feature Selection by Classification Task

Each classification task uses a different label column and requires a different subset of features. The guiding principle is: **do not include features that trivially encode the label**, as this produces classifiers with perfect training accuracy and zero generalization value.

Table 4: Feature selection per classification task.

Task	Label Column	Excluded Features	Rationale for Exclusion
SF Detection	`label_sf`	`spread_factor`, `bw_code`, `mod_code`	Self-reported SF is the answer; BW and mod are correlated configuration parameters.
Modulation Classification	`label_mod`	`mod_code`, `spread_factor`, `bw_code`	Self-reported modulation is the answer; SF/BW are LoRa-specific (leak modulation).
Emitter ID	`label_beacon`	`beacon_id`	Self-reported ID is the answer.

Design Principle: Label Leakage in Cross-Domain Systems

Label leakage is a well-known ML pitfall, but it takes a domain-specific form in this framework. The beacon packet contains self-reported transmission parameters (modulation code, spreading factor, TX power) that the FPGA extracts as feature vector fields. These fields are *also* the ground-truth labels for classification tasks. If a student includes `mod_code` as an input feature when training a modulation classifier, the model will learn to read the self-reported field (a trivial lookup) rather than inferring the modulation from signal characteristics (RSSI, SNR, frequency error, inter-arrival time).

This is more than a data-handling mistake—it reflects a fundamental misunderstanding of what the classifier is supposed to learn. In a real electronic warfare or spectrum monitoring scenario, the emitter does not truthfully report its modulation type. The classifier must infer it from observable signal properties. The feature selection discipline enforced here teaches students to distinguish between "data available in the feature vector" and "data the classifier should be allowed to use as input."

3.5.1 Retained Features by Task

After exclusions, the feature set for each task consists of the signal-derived measurements:

Table 5: Retained features for each classification task.

Feature	Relevance Across Tasks
`rssi`	Primary signal strength indicator; varies with TX power, distance, and path loss.
`snr`	Signal quality metric; affected by modulation type and spreading factor.
`freq_error`	Carrier offset; may carry emitter-specific hardware fingerprint.
`tx_power`	Known TX power (useful context, not a label for SF/mod/ID tasks).
`inter_arrival_ms`	Timing feature; scheduling mode and SF affect on-air time.
`rssi_delta`	Signal stability; emitter-specific fading characteristics.
`obs_count`	Link reliability proxy; not directly predictive but useful as context.
`payload_crc_ok`	Data quality indicator; may correlate with low-SNR conditions.

3.6 Feature Scaling

Different classifier families have different sensitivity to feature scale:

Fit on Training Data Only

The scaler must be `.fit()` on the training set only, then `.transform()` applied to the validation and test sets. Fitting on the entire dataset (including test data) introduces information leakage: the scaler's mean and standard deviation incorporate test-set statistics, giving the model an unfair advantage. This is a subtle but critical methodological point. In scikit-learn, use `Pipeline` to encapsulate the scaler and classifier together to prevent accidental leakage.

Table 6: Scaling strategy by classifier family.

Classifier	Scaling	Rationale
Random Forest	None required	Tree-based models are invariant to monotonic feature transformations.
SVM	StandardScaler	Distance-based kernel requires zero-mean, unit-variance features.
MLP	StandardScaler	Gradient-based training converges faster with normalized inputs.
CNN-1D	StandardScaler	Same as MLP; batch normalization may also be used internally.

3.7 Preprocessing Pipeline Implementation

Figure 1: Preprocessing pipeline. Each stage is a deterministic, reproducible transformation. The scaler is fit on training data only to prevent information leakage.

Exercise: Build the Preprocessing Pipeline

Implement a Python module (`preprocess.py`) that:

1. Loads the CSV and runs all 10 validation checks from Table 2, printing a pass/fail summary.
2. Drops first-observation rows (sentinel handling).
3. Decodes signed fields if the CSV contains raw unsigned values.
4. Provides a function `get_task_data(task, scaler_type)` that returns `(X_train, y_train, X_val, y_val, X_test, y_test)` with the correct feature exclusions and scaling applied. The `task` argument is one of `"sf"`, `"mod"`, `"emitter"`.
5. Exports the fitted scaler as a `.pkl` file for use in the real-time inference script (Week 4).

Verify the pipeline by printing:

- Shape of each split (e.g., `X_train: (1750, 8)`).
- Class distribution in each split for the target task.
- Feature mean and standard deviation after scaling (should be ≈ 0 and ≈ 1 for StandardScaler on training data).

Week 1 Deliverable

Data Preprocessing Report — A document containing:

1. Validation check results (Table 2) with counts of any flagged or removed rows.
2. Sentinel handling decision and number of rows dropped.
3. Signed-field decoding verification (before/after comparison for RSSI and SNR).
4. Feature selection rationale for each classification task, including the label leakage analysis.
5. Preprocessing pipeline source code (`preprocess.py`) with inline documentation.
6. Split summary: sample counts and class distributions for train/val/test for all three tasks.

4 Week 2: Baseline Models and Classical Classifiers

4.1 Objectives

- Establish baseline accuracy using a majority-class classifier and a simple decision tree.
- Train Random Forest classifiers for all three tasks and analyze feature importances.
- Train Support Vector Machine (SVM) classifiers and compare kernel choices.
- Perform hyperparameter tuning using grid search with cross-validation on the training set.
- Evaluate all models on the held-out test set using confusion matrices and per-class metrics.

4.2 Baseline Classifiers

Before training any sophisticated model, students must establish baselines to contextualize results:

Table 7: Baseline classifiers.

Baseline	Implementation	Purpose
Majority Class	`DummyClassifier(strategy="most_frequent")`	Lower bound: any useful model must beat this.
Single Decision Tree	`DecisionTreeClassifier(max_depth=5)`	Interpretable reference; reveals if a single feature suffices.

Why Baselines Matter

For a 6-class SF detection task with balanced classes, the majority-class baseline is ∼16.7% accuracy. For a 2-class modulation task, it is ∼50%. Any classifier that does not substantially exceed these baselines is not learning meaningful patterns. Baselines also reveal class imbalance: if the majority-class baseline for emitter ID is 45% (one beacon contributed disproportionately many samples), this signals a collection imbalance that may need to be addressed through resampling or class weighting.

4.3 Random Forest Classifier

Random Forest is the recommended first "real" classifier for this dataset because it requires no feature scaling, handles mixed feature types naturally, provides built-in feature importance estimates, and is resistant to overfitting on small datasets.

4.3.1 Hyperparameter Search Space

4.3.2 Feature Importance Analysis

After training the Random Forest on each task, extract and visualize the Gini importance (mean decrease in impurity) for each feature:

Table 8: Random Forest hyperparameter grid.

Parameter	Grid Values	Rationale
`n_estimators`	[50, 100, 200, 500]	More trees reduce variance; diminishing returns above 200 for small datasets.
`max_depth`	[5, 10, 20, None]	Controls tree complexity; `None` = fully grown trees.
`min_samples_leaf`	[1, 3, 5, 10]	Regularization; higher values prevent overfitting on noisy features.
`max_features`	["sqrt", "log2", 0.5]	Feature subsampling per split; decorrelates trees.

$$\text{Importance}(f) = \frac{1}{N_{\text{trees}}} \sum_{t=1}^{N_{\text{trees}}} \sum_{n \in \text{splits on } f} p(n) \cdot \Delta G(n) \tag{1}$$

where $p(n)$ is the fraction of samples reaching node n and $\Delta G(n)$ is the Gini impurity reduction at that split.

Expected Feature Importance Patterns

Based on the dataset's physical structure, the following importance patterns are expected:

- **SF Detection:** `inter_arrival_ms` should be the dominant feature, because different spreading factors produce dramatically different on-air times (SF7: ∼50 ms, SF12: ∼1.5 s), directly affecting the inter-packet interval. `snr` may also contribute, as higher SFs improve sensitivity (process gain).
- **Modulation Classification:** `snr` and `inter_arrival_ms` should dominate. LoRa and FSK have fundamentally different demodulation characteristics and on-air times. `freq_error` may differ between modulation types due to different carrier recovery mechanisms in the SX1276.
- **Emitter ID:** `freq_error` and `rssi` are expected to be most informative. Each physical RF module has a slightly different crystal oscillator frequency offset (hardware fingerprint), producing emitter-specific frequency error distributions. `rssi` may differentiate beacons at different distances or TX power settings.

If the observed importances deviate significantly from these expectations, it may indicate a data quality issue, a feature engineering opportunity, or an interesting physical phenomenon worth investigating.

4.4 Support Vector Machine (SVM)

SVM provides a complementary perspective to Random Forest. While RF partitions the feature space with axis-aligned splits, SVM finds optimal separating hyperplanes (or kernelized boundaries) that may capture different decision boundaries.

4.4.1 Hyperparameter Search Space

Table 9: SVM hyperparameter grid.

Parameter	Grid Values	Rationale
`kernel`	["rbf", "linear"]	RBF for nonlinear boundaries; linear for interpretable baseline.
`C`	[0.1, 1, 10, 100]	Regularization strength; higher C fits training data more tightly.
`gamma`	["scale", "auto", 0.01, 0.1]	RBF width; controls how far the influence of a single sample reaches.

SVM Scaling Requirement

SVM performance is highly sensitive to feature scale. Always use the StandardScaler-transformed features (from the preprocessing pipeline) when training SVMs. Using unscaled features will produce a model dominated by whichever feature has the largest numeric range (likely `inter_arrival_ms` at ∼2000 ms vs. `rssi` at ∼-60), effectively ignoring smaller-scale but potentially informative features.

4.5 Evaluation Methodology

All classifiers are evaluated using the same rigorous protocol to ensure fair comparison:

4.5.1 Metrics

Table 10: Evaluation metrics for all classification tasks.

Metric	Definition and Purpose
Overall Accuracy	$\frac{\text{correct predictions}}{\text{total predictions}}$. Single summary metric; can be misleading with class imbalance.
Per-Class Precision	$\frac{\text{TP}_c}{\text{TP}_c+\text{FP}_c}$. Of all predictions for class c, what fraction is correct?
Per-Class Recall	$\frac{\text{TP}_c}{\text{TP}_c+\text{FN}_c}$. Of all true class c samples, what fraction did the model find?
Per-Class F1-Score	$\frac{2\cdot\text{Precision}_c\cdot\text{Recall}_c}{\text{Precision}_c+\text{Recall}_c}$. Harmonic mean; balances precision and recall.
Macro-Average F1	$\frac{1}{C}\sum_{c=1}^{C}\text{F1}_c$. Treats all classes equally regardless of size.
Confusion Matrix	$C\times C$ matrix showing predicted vs. actual class counts. Reveals systematic misclassification patterns.

4.5.2 Evaluation Protocol

1. **Hyperparameter tuning:** Use 5-fold stratified cross-validation on the *training set only* to select optimal hyperparameters. The validation set is used for early stopping (neural networks)

and learning curve analysis, *not* for hyperparameter selection.

2. **Final evaluation:** Train the model with selected hyperparameters on the full training set, then evaluate once on the held-out test set. This single evaluation is the reported result. Do not iteratively tune on the test set.

3. **Statistical context:** For small test sets (∼375 samples at 15% of 2,500), a 2% accuracy difference between models may not be statistically significant. Report 95% confidence intervals using the Wilson score interval for proportions:

$$\mathrm{CI}_{95\%} \approx \hat{p} \pm 1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \tag{2}$$

where $\hat{p}$ is the observed accuracy and n is the test set size.

Exercise: Classical Classifier Comparison

For each of the three classification tasks (SF, modulation, emitter):

1. Train the majority-class baseline and decision tree. Record accuracy.
2. Run grid search for Random Forest (Table 8) using 5-fold CV on training data. Record best parameters and CV accuracy.
3. Run grid search for SVM (Table 9) using 5-fold CV on scaled training data. Record best parameters and CV accuracy.
4. Evaluate all four models on the test set. Produce a comparison table:

Model	**Accuracy**	**Macro F1**	**95% CI**
Majority Class			
Decision Tree			
Random Forest			
SVM (RBF)			

5. Plot the confusion matrix for the best-performing model on each task.
6. Plot the Random Forest feature importance bar chart for each task and compare against the expected patterns.

Week 2 Deliverable

Classical Classifier Training and Evaluation Report — A document containing:

1. Baseline results (majority class, decision tree) for all three tasks.
2. Random Forest: best hyperparameters, CV accuracy, test accuracy, macro F1, confusion matrices, feature importance plots for all three tasks.
3. SVM: best hyperparameters, test accuracy, macro F1, confusion matrices for all three tasks.
4. Model comparison table with 95% confidence intervals.
5. Feature importance analysis: comparison of observed importances against expected patterns, with physical interpretation of any surprises.
6. Source code for training scripts.

5 Week 3: Neural Network Models and Transfer Experiments

5.1 Objectives

- Train Multi-Layer Perceptron (MLP) classifiers for each task and compare against classical baselines.
- Train one-dimensional Convolutional Neural Network (CNN-1D) classifiers that treat the feature vector as a 1D signal.
- Conduct the simulation-to-hardware transfer experiment to quantify the domain gap.
- Analyze classification accuracy as a function of SNR, distance, and beacon count.
- Generate performance-vs.-condition curves that characterize the system's operational envelope.

5.2 Multi-Layer Perceptron (MLP)

The MLP is the simplest neural network architecture applicable to this tabular feature dataset. It serves as the bridge between classical ML (feature engineering + shallow models) and deep learning (end-to-end representation learning).

5.2.1 Architecture

Table 11: MLP architecture specification.

Layer	Input Dim	Output Dim	Notes
Input	—	8	Number of retained features
Dense + ReLU	8	64	First hidden layer
Dropout	64	64	$p = 0.3$; regularization
Dense + ReLU	64	32	Second hidden layer
Dropout	32	32	$p = 0.3$
Dense + Softmax	32	C	Output layer; C = number of classes

5.2.2 Training Configuration

Design Principle: Why Not a Larger Network?

With ∼1,750 training samples and 8 features, the network capacity must be limited to prevent memorization. The 8→64→32→ C architecture has approximately $8 \times 64 + 64 + 64 \times 32 + 32 + 32 \times C + C \approx 2{,}700 + 33C$ trainable parameters. For the 6-class SF task, this is ∼2,900 parameters—roughly 1.7 parameters per training sample, which is at the boundary of overfitting risk. The dropout layers and early stopping provide additional regularization. Students should not increase network depth or width without first demonstrating that the current architecture is underfitting (training loss plateaus above an acceptable level).

Table 12: MLP training hyperparameters.

Parameter	**Value**	**Rationale**
Loss Function	CrossEntropyLoss	Standard for multi-class classification
Optimizer	Adam	Adaptive learning rate; good default for small models
Learning Rate	1×10^{-3}	Standard Adam default
Batch Size	32	Small dataset; smaller batches provide regularization
Max Epochs	200	Upper bound; early stopping typically triggers earlier
Early Stopping	Patience = 15 epochs	Monitor validation loss; restore best weights

5.3 One-Dimensional CNN (CNN-1D)

The CNN-1D architecture treats the 8-element feature vector as a 1D "signal" and applies convolutional filters to learn local feature interactions. While CNNs are typically associated with image or time-series data, applying them to a short feature vector is a useful pedagogical exercise that demonstrates how convolutions operate on structured data.

5.3.1 Architecture

Table 13: CNN-1D architecture specification.

Layer	**Input Shape**	**Output Shape**	**Notes**
Input (reshape)	$(B, 8)$	$(B, 1, 8)$	Add channel dimension; B = batch size
Conv1D + ReLU	$(B, 1, 8)$	$(B, 16, 8)$	16 filters, kernel size 3, padding = 1
Conv1D + ReLU	$(B, 16, 8)$	$(B, 32, 8)$	32 filters, kernel size 3, padding = 1
Global Avg Pool	$(B, 32, 8)$	$(B, 32)$	Pool across the feature dimension
Dense + ReLU	$(B, 32)$	$(B, 32)$	Fully connected hidden layer
Dropout	$(B, 32)$	$(B, 32)$	$p = 0.3$
Dense + Softmax	$(B, 32)$	(B, C)	Output layer

When Does CNN-1D Help?

For an 8-element feature vector, the CNN-1D is unlikely to significantly outperform the MLP—the "signal" is too short for convolutions to capture meaningful spatial patterns. The architecture is included for three pedagogical reasons:

1. It introduces students to the PyTorch CNN API (`nn.Conv1d`, `nn.AdaptiveAvgPool1d`) in a simple context before they encounter it in more natural applications (time-series classification, raw IQ processing).
2. It forces students to reason about tensor shapes—the most common source of bugs in deep learning code.
3. It provides a concrete example of when a more complex architecture does *not* improve performance, which is as important a lesson as when it does.

Students should report the CNN-1D results alongside the MLP and Random Forest and discuss whether the additional complexity is justified.

5.4 Simulation-to-Hardware Transfer Experiment

This experiment is the framework's culminating cross-domain exercise. It quantifies the distribution shift between the engagement simulator's synthetic data and the hardware-collected data from Module 3, measuring how well a model trained in one domain generalizes to the other.

5.4.1 Experimental Protocol

1. **Train on simulation data:** Using the engagement simulator's output (matching the feature vector format where possible), train the best-performing classical classifier (likely Random Forest) on the spreading factor detection task.
2. **Evaluate on hardware data:** Apply the sim-trained model to the Module 3 hardware test set. Record accuracy and confusion matrix.
3. **Train on hardware data:** Train the same classifier type on the Module 3 hardware training set (the standard Week 2 result).
4. **Compare:** The accuracy drop from step 3 to step 2 is the **sim-to-real gap**:

$$\Delta_{\text{sim}\to\text{real}} = \text{Acc}_{\text{hw}\to\text{hw}} - \text{Acc}_{\text{sim}\to\text{hw}} \tag{3}$$

5. **Analyze feature-level shift:** For each feature, compare the distribution (mean, standard deviation, histogram shape) between sim and hardware data. Features with the largest distributional shift are the primary contributors to the accuracy gap.

Expected Results

The sim-to-real accuracy drop is expected to be substantial (15–40% for SF detection) because:

- The simulator uses idealized FSPL propagation; hardware data includes multipath fading, antenna pattern irregularity, and near-field effects.
- The simulator's timing model is exact; hardware inter-arrival times include STM32 interrupt jitter, UART buffering, and RF channel contention.
- The simulator's SNR model is AWGN; real SNR includes colored noise, interference, and receiver non-linearities.

This result is not a failure—it is the *point*. The experiment demonstrates why simulation alone is insufficient for training operational classifiers and motivates the hardware data collection investment in Module 3.

5.5 Performance-vs.-Condition Analysis

This analysis goes beyond aggregate accuracy to characterize *when* the classifier works well and when it degrades, providing an operational envelope for the system.

5.5.1 SNR-Stratified Accuracy

Partition the test set into SNR bins and compute accuracy within each bin:

$$\text{Acc}(\text{SNR} \in [a, b]) = \frac{\text{correct predictions in bin}}{\text{total samples in bin}} \tag{4}$$

Use bins of approximately equal width: $[-20, -10)$, $[-10, -5)$, $[-5, 0)$, $[0, 5)$, $[5, 15]$ dB. Plot accuracy vs. SNR bin midpoint for each classifier.

Expected Pattern

Classification accuracy should generally increase with SNR: at low SNR, features are noisy and less discriminative; at high SNR, signal characteristics are clear and classifiers perform well. The SNR threshold where accuracy drops below an acceptable level (e.g., 80%) defines the system's operational SNR floor. This is directly analogous to the sensitivity specification of the SX1276 receiver from Module 1, but at the ML classification level rather than the demodulation level.

5.5.2 Distance-Stratified Accuracy

Using Campaign C4 data (distance sweep), compute accuracy at each distance. Plot accuracy vs. distance and overlay the RSSI vs. distance curve from Module 3 EDA to correlate classification degradation with signal attenuation.

5.5.3 Beacon-Count-Stratified Accuracy

Using Campaign C5 data, compare classifier accuracy across 1-beacon, 2-beacon, and 3-beacon configurations. This reveals whether the classifier is robust to multi-emitter environments or whether

increasing beacon density degrades performance (e.g., through increased RSSI delta volatility or inter-arrival time irregularity).

Exercise: Performance-vs.-Condition Curves

For the best-performing model on each task:

1. Plot accuracy vs. SNR bin (5 bins, error bars from Wilson CI).
2. Plot accuracy vs. distance (4 distances from C4).
3. Plot accuracy vs. beacon count (1/2/3 from C5).
4. Identify the operating condition where accuracy drops below 80% for each task. Report this as the system's classification floor.
5. Discuss: which degradation mechanism (low SNR, long distance, high beacon count) has the most severe impact on each task? Why?

Week 3 Deliverable

Neural Network Training and Transfer Experiment Report — A document containing:

1. MLP architecture, training curves (loss and accuracy vs. epoch for train and val), final test accuracy, confusion matrices for all three tasks.
2. CNN-1D architecture, training curves, test accuracy, and comparison against MLP (discussion of whether added complexity is justified).
3. Sim-to-real transfer experiment: sim-trained accuracy, hardware-trained accuracy, accuracy gap, feature-level distribution shift analysis.
4. Performance-vs.-condition curves: accuracy vs. SNR, distance, and beacon count with interpretive discussion.
5. Updated model comparison table incorporating all classifiers (baseline, RF, SVM, MLP, CNN-1D) across all tasks.
6. Source code for MLP and CNN-1D training scripts.

6 Week 4: Real-Time Inference and Pipeline Integration

6.1 Objectives

- Deploy a trained classification model in a real-time inference script that classifies live FPGA feature exports.
- Measure inference latency and verify it meets the real-time constraint (classification before the next packet arrives).
- Demonstrate end-to-end operation: beacon transmits → FPGA processes → host classifies → result displayed.
- Conduct final model selection and document the complete ML pipeline.
- Prepare the model and inference script for Module 5 system validation.

6.2 Real-Time Inference Architecture

The real-time inference script is the final link in the end-to-end pipeline. It replaces the Module 3 collection script's role of simply storing feature vectors: instead of writing to CSV, it classifies each incoming feature vector and displays the result.

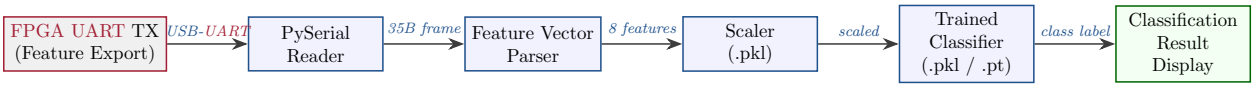

Figure 2: Real-time inference pipeline. The script receives feature vectors from the FPGA via USB-UART, applies the same preprocessing (scaling) used during training, runs the classifier, and displays the result. The scaler and model are loaded from serialized files produced during Weeks 1–3.

6.2.1 Real-Time Constraint

The inference script must classify each feature vector before the next one arrives. The inter-arrival time depends on the beacon configuration:

$$T_{\text{available}} = T_{\text{beacon interval}} - L_{\text{UART}} = T_{\text{beacon}} - 3.73 \text{ ms} \tag{5}$$

For the minimum beacon interval tested (1 second at SF7), $T_{\text{available}} \approx 996$ ms. The inference latency budget is:

Table 14: Inference latency budget.

Component	Expected	Notes
UART frame reception	3.04 ms	35 bytes × 10 bits ÷ 115200 baud
Feature parsing	<0.1 ms	Python struct unpacking
Scaler transform	<0.1 ms	Vectorized NumPy operation
Random Forest inference	<1 ms	scikit-learn predict on 1 sample
MLP inference	<1 ms	PyTorch forward pass (CPU)
Display update	<1 ms	Console print or GUI update
Total	**<6 ms**	Well within the 996 ms budget

Design Principle: Real-Time is Relative

The 996 ms budget makes "real-time" classification trivially achievable on a modern PC. This is by design: the framework's goal is to demonstrate the complete pipeline, not to push inference speed to its limits. In a production system, the latency-critical path would be the FPGA processing (6 clock cycles at 100 MHz = 60 ns), not the host-side Python inference. Students should recognize that the real-time constraint here is soft and generous, and that a production deployment would either run inference on the FPGA itself (using a hardware neural network accelerator) or on an embedded processor with tighter latency requirements. Module 5 explores these architectural alternatives.

6.3 Inference Script Implementation

The inference script (`inference.py`) reuses the feature vector reception code from the Module 3 collection script but replaces the CSV writer with a classification pipeline.

6.3.1 Key Components

1. **Model Loading:** Load the trained classifier (scikit-learn `.pkl` or PyTorch `.pt`) and the fitted scaler (`.pkl`) at startup. For PyTorch models, set the model to evaluation mode (`model.eval()`) and disable gradient computation (`torch.no_grad()`).

2. **Feature Reception:** Receive the 35-byte feature export frame from the FPGA UART TX (Module 3 Table 5), verify Cyclic Redundancy Check (CRC)-8, extract the 32-byte feature vector, and parse it into the 8-element feature array using the same decoding logic from `preprocess.py`.

3. **Classification:** Apply the scaler transform, run the classifier's predict method, and extract the predicted class label and (if available) the prediction probability.

4. **Display:** Print the classification result to the console in a structured format:

```
[12:34:56.789] Beacon 0x01 | RSSI: -62 dBm | SNR: 8.5 dB | Predicted SF: 7 (conf:
0.94)
```

6.3.2 Multi-Task Inference

The script should support running multiple classification tasks simultaneously. Each received feature vector is classified for SF, modulation, and emitter ID in a single pass:

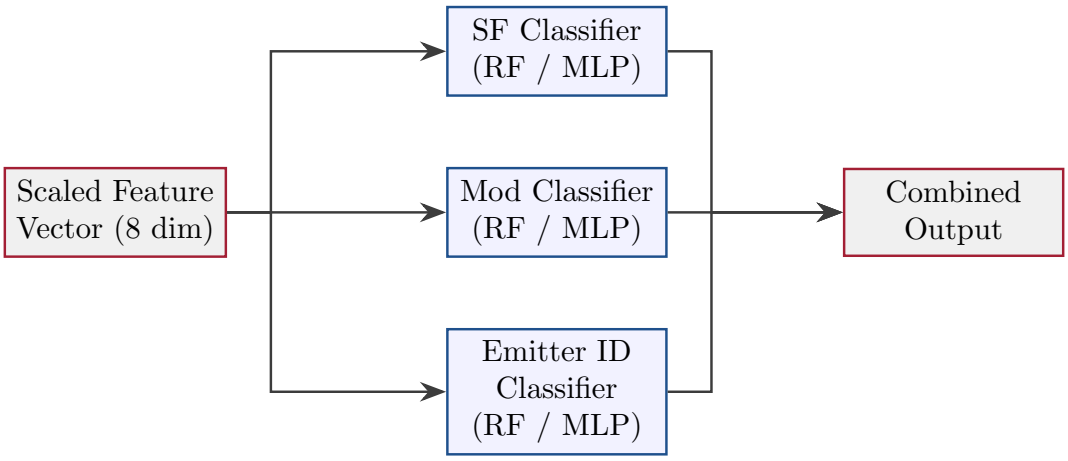

Figure 3: Multi-task inference. A single feature vector is classified by three independent models. Each model uses the task-specific feature subset defined in Table 4.

6.4 Live Demonstration Protocol

Exercise: End-to-End Live Classification

With the full system running (2 beacons, receiver, FPGA, host PC):

1. Start the inference script connected to the FPGA's USB-UART port.
2. Observe real-time classification output for 60 seconds. Verify:
   - SF predictions match the beacons' configured spreading factors.
   - Modulation predictions correctly identify LoRa.
   - Emitter ID predictions distinguish between Beacon 1 and Beacon 2.
3. Change Beacon 1's spreading factor from SF7 to SF10 (requires firmware reflash or runtime reconfiguration). Observe the inference output transition.
4. Move Beacon 2 from 3 m to 8 m distance. Observe whether classification accuracy degrades and whether RSSI changes are reflected in the console output.
5. Introduce a third beacon. Verify that the emitter ID classifier detects the new emitter and the SF classifier correctly identifies its configuration.

Milestone: This demonstration represents the complete end-to-end pipeline: a physical RF signal, transmitted by a beacon, received and processed by an FPGA, and classified by a machine learning model—all operating in real time.

6.5 Inference Latency Measurement

Measure the actual inference latency by timestamping each stage of the pipeline:

1. Record `time.perf_counter()` immediately after UART frame reception.
2. Record `time.perf_counter()` immediately after classification.
3. The difference is the inference processing time (excluding UART reception).

Collect latency measurements for 200 consecutive classifications and report the mean, standard deviation, and 99th percentile. Verify that the 99th percentile is well below $T_{\text{available}}$.

6.6 Final Model Selection

Based on the complete results from Weeks 2–4, select the final model for each task. The selection criteria should balance:

Table 15: Model selection criteria.

Criterion	Weight	Justification
Test Accuracy / Macro F1	40%	Primary performance metric.
Robustness across SNR range	25%	Model should degrade gracefully, not catastrophically.
Inference Latency	15%	Must fit within the real-time budget.
Interpretability	10%	Feature importances aid system understanding.
Complexity / Maintenance	10%	Simpler models are preferred if accuracy is comparable.

Expected Outcome

For this dataset size (∼2,500 samples, 8 features), Random Forest is likely to be the best overall choice: it typically matches or exceeds the neural network models on tabular data at this scale, provides feature importances, requires no scaling, and has negligible inference latency. The MLP may slightly outperform on tasks with complex feature interactions, while the CNN-1D is unlikely to improve over the MLP given the short feature vector length. The primary value of the neural network experiments is pedagogical: students learn to implement, train, and evaluate them, compare them against simpler alternatives, and make an evidence-based model selection rather than defaulting to "deep learning is always better."

Week 4 Deliverable

Real-Time Inference and Final Pipeline Report — A document containing:

1. Inference script source code (`inference.py`) with documentation.
2. Live demonstration results: classification output log for the 60-second run, including the SF change and distance change experiments.
3. Inference latency analysis: histogram of processing times, mean/std/99th percentile, comparison against real-time budget.
4. Final model selection table: all classifiers × all tasks, with accuracy, macro F1, robustness rating, latency, and final selection with justification.
5. Complete ML pipeline documentation: end-to-end flow from CSV to real-time classification, including all file paths, dependencies, and execution instructions.
6. Serialized model files (`.pkl` / `.pt`) and scaler files, ready for Module 5 system validation.

7 Complete Classification Pipeline Architecture

Figure 4 shows the complete ML pipeline as developed across Module 4, from data ingestion to real-time inference.

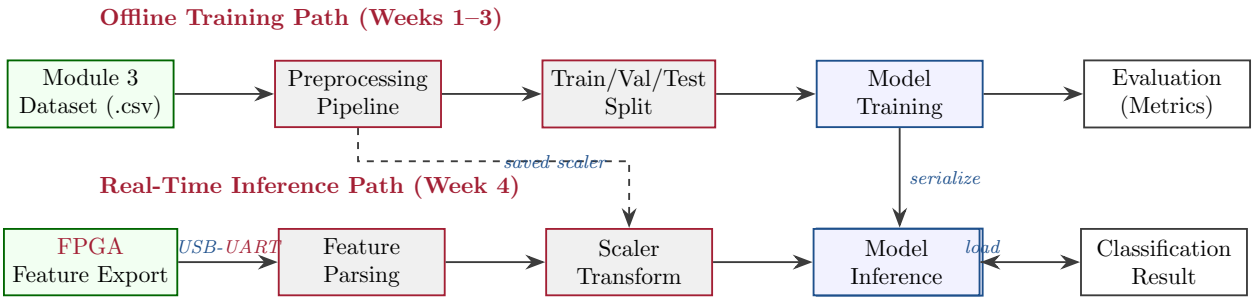

Figure 4: Complete Module 4 ML pipeline. The offline path (top) trains and evaluates classifiers on the Module 3 dataset. The real-time path (bottom) deploys the trained model to classify live FPGA feature exports. The scaler and model artifacts connect the two paths.

8 Software Reference

Table 16: Required software and libraries for Module 4.

Package	Version	Purpose
Python	3.10+	Runtime environment
`pandas`	≥2.0	Dataset loading, manipulation, and CSV I/O
`numpy`	≥1.24	Numerical operations, array manipulation
`scikit-learn`	≥1.3	Classical classifiers (RF, SVM), metrics, preprocessing
`matplotlib`	≥3.7	Plotting (confusion matrices, learning curves, bar charts)
`seaborn`	≥0.12	Statistical visualizations (heatmaps, violin plots)
`torch` (PyTorch)	≥2.0	Neural network models (MLP, CNN-1D)
`pyserial`	≥3.5	Real-time UART communication with FPGA
`joblib`	≥1.3	Model and scaler serialization (.pkl files)
Terminal Emulator	—	UART monitoring (PuTTY, Tera Term)

8.1 Hardware Requirements

Table 17: Hardware required for Module 4.

Item	Qty	Purpose
STM32 Nucleo-L476RG + RFM95W	≥ 2	Beacons for live inference demonstration
STM32 Nucleo-L476RG + RFM95W	1	Receiver MCU
Digilent Nexys A7-100T	1	FPGA running Module 2 pipeline
USB Micro-B Cables	4–5	Power, programming, UART
Host PC (any OS)	1	Python runtime, model training, inference

9 Assessment Rubric

Table 18: Module 4 assessment rubric.

Assessment Item	Weight	Due
Data Preprocessing Report (validation, sentinels, decoding, feature selection, pipeline code)	15%	Week 1
Classical Classifier Report (baselines, RF, SVM, tuning, confusion matrices, feature importances)	25%	Week 2
Neural Network and Transfer Report (MLP, CNN-1D, sim-to-real gap, performance-vs.-condition)	30%	Week 3
Real-Time Inference and Final Pipeline Report (inference script, live demo, latency, model selection)	30%	Week 4

10 Recommended Resources

10.1 Machine Learning Textbooks

1. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O'Reilly, 2022. (Chapters 2–3: classification pipeline, model evaluation, confusion matrices. Chapter 10: neural networks with Keras/PyTorch.)

2. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. (Chapters 5–6: neural networks, backpropagation, regularization.)

3. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009. (Chapter 15: Random Forests. Chapter 12: SVMs.)

10.2 RF and Signal Classification References

1. T. J. O'Shea, T. Roy, and T. C. Clancy, "Over-the-Air Deep Learning Based Radio Signal Classification," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 168–179, Feb. 2018.

2. S. Rajendran *et al.*, "Deep Learning Models for Wireless Signal Classification With Distributed Low-Cost Spectrum Sensors," *IEEE Transactions on Cognitive Communications and Networking*, vol. 4, no. 3, pp. 433–445, Sep. 2018.

3. K. Merchant *et al.*, "Deep Learning for RF Device Fingerprinting in Cognitive Communication Networks," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 160–167, Feb. 2018.

10.3 Software Documentation

1. scikit-learn User Guide. [Online]. Available: https://scikit-learn.org/stable/user_guide.html (Classification, model evaluation, preprocessing, pipelines.)

2. PyTorch Tutorials. [Online]. Available: https://pytorch.org/tutorials/ (Neural network training, data loading, model serialization.)

3. pandas Documentation. [Online]. Available: https://pandas.pydata.org/docs/ (DataFrame operations, CSV I/O, data cleaning.)

4. matplotlib Gallery. [Online]. Available: https://matplotlib.org/stable/gallery/ (Confusion matrix plotting, bar charts, learning curves.)

10.4 Domain Adaptation and Distribution Shift

1. J. Quinonero-Candela *et al.*, *Dataset Shift in Machine Learning*, MIT Press, 2009. (Foundational treatment of covariate shift, prior probability shift, and concept drift.)

2. K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A Survey of Transfer Learning," *Journal of Big Data*, vol. 3, no. 9, 2016.

11 Transition to Module 5

Upon successful completion of Module 4, students will have:

- A reproducible preprocessing pipeline that transforms raw FPGA feature vectors into classifier-ready input, with explicit handling of sentinels, signed fields, feature selection, and scaling.
- Trained and evaluated classifiers for three distinct tasks (spreading factor detection, modulation classification, emitter identification) using both classical ML (Random Forest, SVM) and neural network (MLP, CNN-1D) approaches.
- Quantitative evidence of the simulation-to-hardware distribution shift, measured through cross-domain transfer accuracy degradation and feature-level distributional analysis.
- Performance-vs.-condition curves that characterize classifier accuracy as a function of SNR, physical distance, and beacon count, defining the system's operational classification envelope.
- A real-time inference script that classifies live FPGA feature exports, demonstrating end-to-end pipeline operation from RF transmission to classification output.
- Serialized model and scaler artifacts ready for deployment and benchmarking.

Module 5 will subject the complete system to stress testing and final validation. The trained classifiers from this module will be evaluated under adversarial conditions: concurrent multi-beacon operation at maximum density, rapid parameter changes, edge-of-range deployments, and extended-duration stability tests. The performance-vs.-condition analysis from Week 3 provides the baseline expectations; Module 5 will determine whether the system meets those expectations under sustained operational load. The final project report and presentation will synthesize results from all five modules into a coherent narrative of cross-domain system design, integration, and evaluation.

Glossary

BRAM	Block Random Access Memory	5
CRC	Cyclic Redundancy Check	21
FPGA	Field-Programmable Gate Array	1, 2, 3, 5, 7, 20, 21, 22, 25, 26, 29
FSK	Frequency Shift Keying	4, 11
MCU	Microcontroller Unit	26
ML	Machine Learning	3, 7, 15, 18, 20, 24, 25, 29
OOK	On-Off Keying	4
RF	Radio Frequency	1, 2, 11, 18, 22, 29
RSSI	Received Signal Strength Indicator	5, 6, 7, 9, 19, 22
SNR	Signal-to-Noise Ratio	i, 1, 3, 5, 7, 9, 15, 18, 19, 23, 29
UART	Universal Asynchronous Receiver/Transmitter	1, 3, 5, 18, 20, 21, 22, 23, 25, 26

End of Module 4 Lesson Plan